

Assignment 3: Derivations, Typechecker, and Language Extensions

CSC324H1, Fall 2025

Due Date: October 24, 2025 (11:59 pm)

In this assignment, you will:

- Get more familiar with formal semantics, evaluation strategies, and typing derivations;
- Implement a typechecker for the simple language we have been working on;
- Implement simple extensions to the language, including records, pairs, and sums.

Please read the instructions carefully before you start.

Instructions

For Part 1, please submit a PDF file. You are encouraged to use $\text{\LaTeX}$ to write your solutions. A $\text{\LaTeX}$ template with typesetting examples and guidance has been provided to you. Typing in pure text or handwritten solutions are acceptable. You are responsible for making sure that your solutions are easily legible, so we can mark your solutions correctly and provide feedback to you.

For Part 2, a starter project with code skeleton is provided to you. You should complete the functions as required later in the handout and marked with `error "TODO"` in the code. Run the tests provided to you to check your work. Note that your code will be graded based on additional hidden tests.

Please note that for all assignments:

- It is recommended to only submit all required files as specified on MarkUs.
- You are responsible for making sure that your code has no syntax errors or compile *errors*. If your file cannot be imported by another file, you may receive a grade of zero. Compiler *warnings* can help you locate potential issues, but they do not affect your grade.
- If you're not intending to complete one of the functions, do *not* remove the function signature. If you are including a partial solution, make sure it doesn't cause a compile error. If your partial solution causes compile errors, it's better to comment out your solution (but not the signature).
- Do not change the `module ... where` and `import ...` lines of your code. We may not be able to mark your submission if these lines are modified.

Part 1: Formal Semantics and Typing Derivations

(50 pts)

For this part, you will practice writing evaluation and typing derivations for the untyped and simply-typed lambda calculi with extensions.

(a)

(30 pts)

Consider the following term in the untyped lambda calculus:

$$(\lambda y. (\lambda x. y \text{ true}) \text{ true}) ((\lambda x. \lambda y. x) \text{ false})$$

Evaluate the term down to normal form using each of the following set of rules:

1. *call-by-value*, *small-step* evaluation rules introduced in the Week 4 lecture.
2. *call-by-name*, *small-step* evaluation rules introduced in the Week 4 lecture.
3. *environment-based*, *big-step* evaluation rules introduced in the Week 5 lecture. Start with the empty environment.

Formatting instructions:

For the small-step semantics, start with the original term, then (1) write down the result after each step, and (2) annotate each step with the evaluation rule used for that step. Your presentation should be similar to the examples on slides 15 and 16 of the Week 4 lecture.

For the big-step semantics, write down the big-step evaluation derivation tree, similar to the one on slide 40 of the Week 5 lecture in style.

(b)

(20 pts)

Consider the following term, and the evaluation and typing rules for the simply-typed lambda calculus in Figure 9-1 of TAPL with booleans in Figures 3-1, 8-1 and its extension of let bindings in Figure 11-4 and lists in Figure 11-13:

$$\text{let } x = (\text{cons}[\text{Bool}] \text{ false nil}[\text{Bool}]) \text{ in} \\ \text{if isnil}[\text{Bool}] x \text{ then true else false}$$

Finish the following tasks:

1. Write down the equivalent term without explicit use of let, expressing it instead with the derived form of let. You may need to insert a type annotation as appropriate.
2. Write down the typing derivation for the original term, similar in style to the ones on slides 42 and 43 of the Week 5 lecture. Start from the empty typing context $\emptyset$.
3. Evaluate the original term down to normal form using the *call-by-value* and *small-step* semantics. Write down the evaluation result after each step and the corresponding evaluation rules used, presented in style similar to (a)1.

Part 2: Typechecker and Extensions

(50 pts)

For this part, we will extend the simple language we have been working on in the previous assignments with records, pairs, and sums. You will then implement a typechecker for the extended language, and extend the evaluator from the previous assignments to handle the new language constructs.

(a) Typechecking and Language Extensions

Type Syntax The syntax of types is defined in `Type.hs` as follows:

```
data Type = TyBool | TyNat | TyArrow Type Type
          | TyRecord [(String, Type)]
          | TyProd Type Type | TySum Type Type
```

Language Extensions We extend the syntax of the language from Assignment 2 in the *nameless representation*, i.e. `Term` in `Term.hs`, to support new language features:

```
-- Nameless representation of terms
data Term = ... -- existing cases
           ----- annotated lambdas
           | TmAnnAbs Type Term
           ----- records
           | TmRecord [(String, Term)] | TmProj Term String
           ----- pairs
           | TmPair Term Term | TmFst Term | TmSnd Term
           ----- sums
           | TmInl Term Type | TmInr Term Type | TmMatch Term Term Term
```

Auxiliary Definitions We also introduce a few auxiliary types and datatypes in `TypeCheck.hs`:

```
type TcContext = [Type]

data TcResult = TcOk Type
              | TcError String

lookupCtx :: Int -> TcContext -> Maybe Type
```

Name	Note
TcContext	Typing context as a list of types, where the position in the list corresponds to the de Bruijn index of a variable.
TcResult	Denotes whether a term typechecks.
TcOk	Indicates successful typechecking with the type as the output.
TcError	Indicates a type error with an error message.
lookupCtx	Typing context lookup function.

Typechecking The typechecking function is declared in `TypeCheck.hs` with the following signature:

```
typeCheck :: TcContext -> Term -> TcResult
```

The typechecker takes in a typing context `TcContext` and a term to type, and returns a result `TcResult`, which is either `TcOk` Type containing the type of the term, or `TcError` containing an error message.

(b) Small-Step Evaluation of Language Extensions

Extend the small-step evaluator from Assignment 2 to support the new language features by extending the `oneStep` function in `SmallStep.hs` and the `shifting` and `subst` functions in `DeBruijn.hs`. Consider carefully the interactions between the nameless representation and the new language constructs.

Your Tasks and Tips

1. (30 pts) Implement the typechecking function `typeCheck` in `TypeCheck.hs`.
 - The typechecker should essentially implement the typing rules as listed in Figures 8-1, 8-2, and 9-1 of TAPL for the basic arithmetic and lambda calculus terms, and Figures 11-5, 11-7, and 11-10 for the new language constructs.
 - You might find Chapter 10 of TAPL useful.
 - Error messages are not tested.
2. (20 pts) Extend `shifting` and `subst` in `DeBruijn.hs` and the one-step evaluator `oneStep` in `SmallStep.hs` to support the new language features of records, pairs, and sums.
 - The extensions should essentially implement the evaluation rules as listed in Figures 11-5, 11-7, and 11-10 of TAPL.
 - *Do not* modify the given `multiStep`. However, make sure `multiStep` is correct with respect to the new specifications and your new implementation of `oneStep`.
 - Error messages are not tested.